Jobsheet 6

Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom

Disusun oleh:
Mukhammad Raffi Zabra

T1 – 1G

254107020059

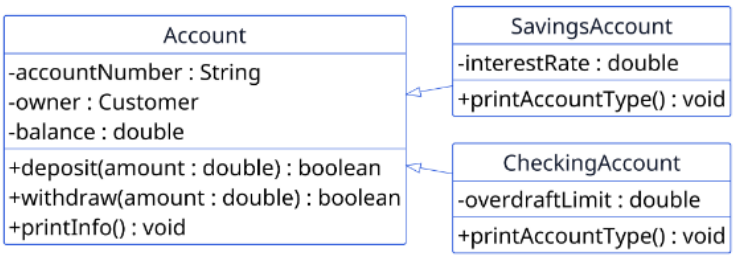
POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141

TAHUN 2026-2027

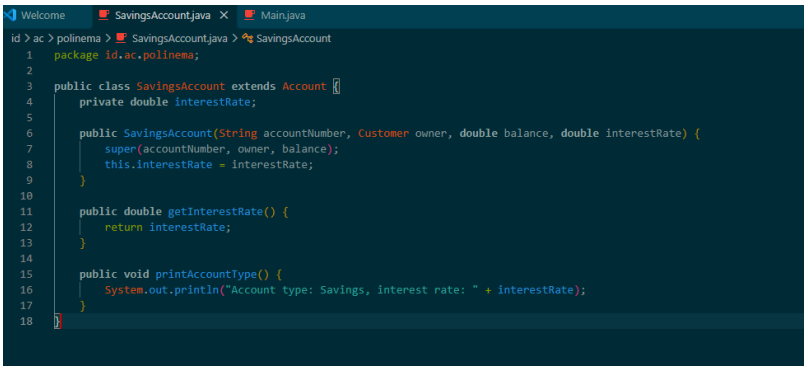
C. Langkah Kerja

Langkah 1: SavingsAccount, Subclass Pertama



Gambar 1 Diagram

Class SavingsAccount.java



```java
package id.ac.polinema;

public class SavingsAccount extends Account {
    private double interestRate;

    public SavingsAccount(String accountNumber, Customer owner, double balance, double interestRate) {
        super(accountNumber, owner, balance);
        this.interestRate = interestRate;
    }

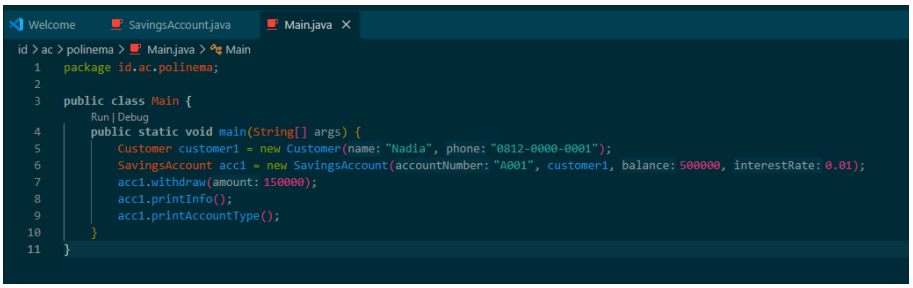
    public double getInterestRate() {
        return interestRate;
    }

    public void printAccountType() {
        System.out.println("Account type: Savings, interest rate: " + interestRate);
    }
}
```

Gambar 2 SavingsAccount.java

Penjelasan :
Kode ini mendefinisikan kelas SavingsAccount sebagai *subclass* dari Account menggunakan kata kunci extends. Kelas ini memiliki atribut tambahan interestRate bertipe double dengan akses *private*. *Constructor* SavingsAccount menggunakan super() untuk memanggil *constructor* dari *superclass* Account, lalu dilanjutkan dengan *method getter* getInterestRate() serta *method* printAccountType() untuk menampilkan informasi suku bunga.

Class Main.java



```java
package id.ac.polinema;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Customer customer1 = new Customer(name: "Nadia", phone: "0812-0000-0001");
        SavingsAccount acc1 = new SavingsAccount(accountNumber: "A001", customer1, balance: 500000, interestRate: 0.01);
        acc1.withdraw(amount: 150000);
        acc1.printInfo();
        acc1.printAccountType();
    }
}
```

Gambar 3 Main.java

Penjelasan :
Pada kelas Main, dibuat objek customer1 dan objek acc1 bertipe SavingsAccount dengan nomor rekening "A001", saldo awal 500000, serta suku bunga 0.01. Program kemudian melakukan penarikan dana sebesar 150000 menggunakan *method* warisan withdraw(), lalu memanggil printInfo() dan printAccountType() untuk menampilkan sisa saldo beserta tipe rekening.

> ✅ **Checkpoint:** program mencetak `A001 - Nadia - balance: 350000.0` (dari `printInfo()`, method warisan yang tidak ditulis ulang) diikuti `Account type: Savings, interest rate: 0.01` (dari `printAccountType()`, method baru milik SavingsAccount).

> ⚠ **Jika gagal:** apabila muncul error `constructor Account in class Account cannot be applied to given types`, periksa apakah `super(...)` di SavingsAccount dipanggil dengan urutan dan jumlah argumen yang persis sama dengan constructor Account yang dituju.

hasil :

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6> javac id\ac\polinema\*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6> java id.ac.polinema.main
Error: Could not find or load main class id.ac.polinema.main
Caused by: java.lang.NoClassDefFoundError: id/ac/polinema/main (wrong name: id/ac/polinema/Main)
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6> java id.ac.polinema.Main
A001 - Nadia - balance: 350000.0
Account type: Savings, interest rate: 0.01
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6>
```

Gambar 4 Hasil Terminal / Checkpoint 1

Langkah 2: CheckingAccount, Subclass Kedua

Class CheckingAccount.java

```java
package id.ac.polinema;

public class CheckingAccount extends Account {
    private double overdraftLimit;

    public CheckingAccount(String accountNumber, Customer owner, double balance, double overdraftLimit) {
        super(accountNumber, owner, balance);
        this.overdraftLimit = overdraftLimit;
    }

    public double getOverdraftLimit() {
        return overdraftLimit;
    }

    public void printAccountType() {
        System.out.println("Account type: Checking, overdraft limit: " + overdraftLimit);
    }
}
```

Gambar 5 CheckingAccount.java

Penjelasan :
Kode ini mendefinisikan kelas CheckingAccount yang mewarisi kelas Account. Kelas ini memiliki atribut baru overdraftLimit bertipe double untuk menyimpan batas penarikan lebih. *Constructor* memanfaatkan super() untuk inisialisasi data induk Account, serta menyediakan *method getter* getOverdraftLimit() dan *method* printAccountType() untuk menampilkan informasi limit *overdraft*.

Class Main.java

```java
package id.ac.polinema;

public class Main {
    Run | Debug
    public static void main(String[] args) {
        Customer customer1 = new Customer(name: "Nadia", phone: "0812-0000-0001");
        SavingsAccount acc1 = new SavingsAccount(accountNumber: "A001", customer1, balance: 500000, interestRate: 0.01);
        acc1.withdraw(amount: 150000);
        acc1.printInfo();
        acc1.printAccountType();

        Customer customer2 = new Customer(name: "Sari", phone: "0812-0000-0002");
        CheckingAccount acc2 = new CheckingAccount(accountNumber: "A002", customer2, balance: 200000, overdraftLimit: 50000);
        acc2.withdraw(amount: 230000);
        acc2.printInfo();
        acc2.printAccountType();
    }
}
```

Gambar 6 Main.java

Penjelasan :
Kode Main.java diperbarui dengan menambahkan objek customer2 dan acc2 bertipe CheckingAccount dengan saldo 200000 dan limit *overdraft* 50000. Dilakukan percobaan penarikan sebesar 230000 melalui acc2.withdraw(230000) sebelum memanggil printInfo() dan printAccountType() untuk kedua rekening.

> ☑ **Checkpoint:** program mencetak empat baris: dua baris pertama untuk acc1 sama seperti Langkah 1, diikuti `A002 - Sari - balance: 200000.0` dan `Account type: Checking, overdraft limit: 50000.0`. Saldo acc2 tetap 200000.0 karena penarikan 230000 ditolak: `withdraw()` yang diwarisi dari Account hanya mengizinkan penarikan sebesar saldo yang tersedia, dan belum tahu cara memakai `overdraftLimit`. Ini bukan error, melainkan hal
>
> yang sengaja diamati: atribut baru saja tidak cukup, subclass juga butuh cara menulis ulang perilaku yang diwarisi, dan itulah yang diselesaikan lewat overriding pada Pertemuan 7.
>
> ⚠ **Jika gagal:** apabila `overdraftLimit` tidak pernah tersimpan dengan benar, periksa apakah nama parameter constructor tidak keliru tertukar urutannya dengan balance.

hasil :

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6> javac id\ac\polinema\*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6> java id.ac.polinema.Main
A001 - Nadia - balance: 350000.0
Account type: Savings, interest rate: 0.01
A002 - Sari - balance: 200000.0
Account type: Checking, overdraft limit: 50000.0
```

Gambar 7 Hasil Terminal / Checkpoint 2

Langkah 3: Bank Mini Mengelola Beberapa Jenis Rekening

Class Main.java

```
Welcome | SavingsAccount.java | Main.java | Bank.java | Customer.java | CheckingAccount.java
id > ac > polinema > Main.java > Main
1  package id.ac.polinema;
2
3  public class Main {
       Run | Debug
4      public static void main(String[] args) {
5          Customer customer1 = new Customer(name: "Nadia", phone: "0812-0000-0001");
6          SavingsAccount acc1 = new SavingsAccount(accountNumber: "A001", customer1, balance: 500000, interestRate: 0.01);
7          acc1.withdraw(amount: 150000);
8
9          Customer customer2 = new Customer(name: "Sari", phone: "0812-0000-0002");
10         CheckingAccount acc2 = new CheckingAccount(accountNumber: "A002", customer2, balance: 200000, overdraftLimit: 50000);
11         acc2.withdraw(amount: 230000);
12
13         Bank bank = new Bank(capacity: 10);
14         bank.addAccount(acc1);
15         bank.addAccount(acc2);
16         bank.printAllAccount();
17     }
18 }
```

Gambar 8 Main.java

Penjelasan :
Kode Main.java memuat pembuatan objek bank bertipe Bank dengan kapasitas 10. Berbagai jenis rekening (acc1 yang bertipe SavingsAccount dan acc2 yang bertipe CheckingAccount) dimasukkan ke dalam objek bank menggunakan *method* bank.addAccount(), kemudian seluruh rekening ditampilkan menggunakan bank.printAllAccount().

> ✅ **Checkpoint:** `printAllAccounts()` mencetak `A001 - Nadia - balance: 350000.0` diikuti `A002 - Sari - balance: 200000.0`, kedua baris memakai bentuk yang sama persis karena `printInfo()` belum ditulis ulang oleh subclass mana pun. Bank belum bisa menampilkan bunga atau limit overdraft masing-masing jenis rekening lewat `printAllAccounts()`.

> ⚠ **Jika gagal:** apabila muncul error `incompatible types` saat `addAccount(acc1)` dipanggil, periksa apakah parameter `addAccount` di Bank bertipe `Account`, bukan tipe subclass tertentu.

Hasil :

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6> javac id\ac\polinema\*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6> java id.ac.polinema.Main
A001 - Nadia - balance: 350000.0
A002 - Sari - balance: 200000.0
```

Gambar 9 Hasil Terminal / Checkpoint 3

D. Tugas dan Hasil Kerja
Kumpulkan hal berikut sesuai format yang diminta Dosen:
• Screenshot output program setelah Langkah 3.

```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6> javac id\ac\polinema\*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6> java id.ac.polinema.Main
A001 - Nadia - balance: 350000.0
A002 - Sari - balance: 200000.0
```

Gambar 10 Hasil Terminal / Checkpoint 3

• Tugas mandiri:
Class BussinesAccount.java

Welcome | SavingsAccount.java | Main.java | BusinessAccount.java | Bank.java | Customer.java | CheckingAcco

id > ac > polinema > BusinessAccount.java > BusinessAccount

```java
package id.ac.polinema;

public class BusinessAccount extends Account {
    private double monthlyTransactionFee;

    public BusinessAccount(String accountNumber, Customer owner, double balance, double monthlyTransactionFee) {
        super(accountNumber, owner, balance);
        this.monthlyTransactionFee = monthlyTransactionFee;
    }

    public double getMonthlyTransactionFee() {
        return monthlyTransactionFee;
    }

    public void printAccountType() {
        System.out.println("Account type: Business, monthly fee: " + monthlyTransactionFee);
    }
}
```

Gambar 11 BusinessAccount.java

Penjelasan :
Kode ini membuat *subclass* ketiga yaitu BusinessAccount yang ditujukan untuk nasabah korporat. Kelas ini menambah atribut monthlyTransactionFee bertipe double, *method getter* getMonthlyTransactionFee(), dan *method* printAccountType() untuk mencetak biaya transaksi bulanan.

Class Main.java



```java
package id.ac.polinema;

public class Main {
    public static void main(String[] args) {
        Customer customer1 = new Customer(name: "Nadia", phone: "0812-0000-0001");
        SavingsAccount acc1 = new SavingsAccount(accountNumber: "A001", customer1, balance: 500000, interestRate: 0.01);
        acc1.withdraw(amount: 150000);

        Customer customer2 = new Customer(name: "Sari", phone: "0812-0000-0002");
        CheckingAccount acc2 = new CheckingAccount(accountNumber: "A002", customer2, balance: 200000, overdraftLimit: 50000);
        acc2.withdraw(amount: 230000);

        Customer customer3 = new Customer(name: "Budi", phone: "0812-0000-0003");
        BusinessAccount acc3 = new BusinessAccount(accountNumber: "A003", customer3, balance: 1000000, monthlyTransactionFee… 15000.0);

        Bank bank = new Bank(capacity: 10);
        bank.addAccount(acc1);
        bank.addAccount(acc2);
        bank.addAccount(acc3);

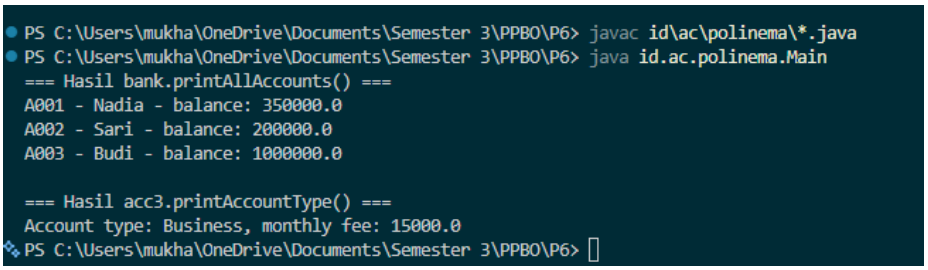
        System.out.println("=== Hasil bank.printAllAccounts() ===");
        bank.printAllAccount();

        System.out.println("\n=== Hasil acc3.printAccountType() ===");
        acc3.printAccountType();
    }
}
```

Gambar 12 Main.java

Penjelasan :
Kode Main.java diperbarui untuk menguji ketiga jenis rekening (SavingsAccount, CheckingAccount, dan BusinessAccount acc3 milik Budi). Ketiga objek dimasukkan ke dalam objek bank, dilanjutkan dengan pengujian pemanggilan bank.printAllAccount() serta acc3.printAccountType().

Hasil :



```
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6> javac id\ac\polinema\*.java
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6> java id.ac.polinema.Main
=== Hasil bank.printAllAccounts() ===
A001 - Nadia - balance: 350000.0
A002 - Sari - balance: 200000.0
A003 - Budi - balance: 1000000.0

=== Hasil acc3.printAccountType() ===
Account type: Business, monthly fee: 15000.0
PS C:\Users\mukha\OneDrive\Documents\Semester 3\PPBO\P6>
```

Gambar 13 Hasil Terminal - Tugas Mandiri

2. Jawab secara singkat (2-3 kalimat): Bank.printAllAccounts() belum bisa menampilkan info khusus tiap jenis rekening (bunga, limit overdraft, atau biaya bulanan), padahal method printAccountType() sudah ada di setiap subclass. Mengapa demikian, dan apa yang menurutmu perlu diubah agar printAllAccounts() bisa menampilkannya secara otomatis?
Jawaban :
Hal tersebut terjadi karena method Bank.printAllAccounts() memanggil printInfo() yang didefinisikan pada superclass Account, di mana kelas Account belum mengenai method printAccountType() maupun spesifikasi khusus subclass. Agar dapat menampilkannya secara otomatis, method printInfo() pada Account dapat di-*override* di masing-masing subclass (SavingsAccount, CheckingAccount, BusinessAccount) dengan menyertakan pemanggilan printAccountType() di dalamnya sehingga memanfaatkan mekanisme *polymorphism*.

3. Jawab secara singkat (2-3 kalimat): apakah SavingsAccount sebaiknya dipecah lagi menjadi subclass (misalnya jenis dengan bunga tetap dan jenis dengan bunga berjen-jang)? Beri satu contoh subclass yang menurutmu masuk akal beserta atribut barunya, atau jelaskan mengapa pemecahan lebih lanjut belum diperlukan untuk Bank Mini.
Jawaban :
Untuk kebutuhan skala aplikasi Bank Mini saat ini, pemecahan SavingsAccount lebih lanjut belum diperlukan agar struktur program tidak terlalu kompleks (*over-engineering*). Namun, jika sistem diperluas untuk mendukung variasi jenis tabungan, pemecahan menjadi subclass seperti DepositAccount (Tabungan Berjangka) masuk akal dengan menambahkan atribut baru lockPeriodMonths (durasi kunci tabungan dalam bulan).